Day 9: File Handling in Python (Detailed Notes for Data Analysis Students)

What is File Handling?

File handling allows Python to:

- Read data from files
- Write data to files
- Update existing files
- Store reports
- Save processed data

Instead of keeping data only in memory, file handling lets you work with permanent data stored on disk.

Why is File Handling Important?

Data Analysts work with files every day, such as:

- CSV files
- Excel files
- Text files
- Log files
- JSON files

Common tasks include:

- Reading customer data
- Cleaning data
- Generating reports
- Saving analysis results
- Processing multiple files

Types of Files

TEXT FILE (.TXT)

Example:

```
Rahul
Amit
Priya
```

CSV FILE (.CSV)

```
Name,Salary
Rahul,50000
Amit,60000
Priya,55000
```

CSV is one of the most common formats used in Data Analysis.

Opening a File

Syntax:

```
file = open("filename.txt", "mode")
```

File Modes

Mode	Description
`r`	Read file
`w`	Write (creates or overwrites)
`a`	Append to file
`x`	Create new file

`r+`	Read and write
`w+`	Write and read

Read a File

Suppose **employees.txt** contains:

```
Rahul
Amit
Priya
```

Code:

```
file = open("employees.txt", "r")

print(file.read())

file.close()
```

Output

```
Rahul
Amit
Priya
```

Read One Line

```
file = open("employees.txt", "r")

print(file.readline())

file.close()
```

Output

```
Rahul
```

Read All Lines

```
file = open("employees.txt", "r")

print(file.readlines())

file.close()
```

Output

```
['Rahul\n', 'Amit\n', 'Priya']
```

Read Using Loop

```
file = open("employees.txt", "r")

for line in file:
    print(line.strip())

file.close()
```

Output

```
Rahul
Amit
Priya
```

Write to a File

```
file = open("students.txt", "w")

file.write("Rahul")

file.close()
```

The file is created if it doesn't exist.

⚠️ **Note:** `w` mode overwrites the entire file.

Write Multiple Lines

```
file = open("students.txt", "w")

file.write("Rahul\n")
file.write("Amit\n")
file.write("Priya\n")

file.close()
```

Append Data

```
file = open("students.txt", "a")

file.write("Neha\n")

file.close()
```

Output in file:

```
Rahul
Amit
Priya
Neha
```

Using `with` Statement (Recommended)

The `with` statement automatically closes the file.

```
with open("employees.txt", "r") as file:

    print(file.read())
```

No need to call `close()`.

Check if File Exists

```
import os

if os.path.exists("employees.txt"):
    print("File Found")
```

```
else:
    print("File Not Found")
```

Delete a File

```
import os

os.remove("employees.txt")
```

Always check if the file exists before deleting it.

Exception Handling

```
try:

    file = open("abc.txt", "r")

    print(file.read())

except FileNotFoundError:

    print("File does not exist")
```

Output

```
File does not exist
```

Working with CSV Files

Python provides the `csv` module for handling CSV files.

READ CSV

Suppose **employees.csv** contains:

```
Name,Salary
Rahul,50000
Amit,60000
Priya,55000
```

Code:

```
import csv

with open("employees.csv", "r") as file:

    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Output

```
['Name', 'Salary']
['Rahul', '50000']
['Amit', '60000']
['Priya', '55000']
```

WRITE CSV

```
import csv

with open("employees.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow(["Name", "Salary"])
    writer.writerow(["Rahul", 50000])
    writer.writerow(["Amit", 60000])
```

Real Data Analyst Examples

EXAMPLE 1: READ EMPLOYEE FILE

```
with open("employees.txt", "r") as file:

    for employee in file:
        print(employee.strip())
```

EXAMPLE 2: SAVE SALES REPORT

```
sales = [10000, 20000, 30000]
```

```
with open("report.txt", "w") as file:

    file.write("Total Sales = " + str(sum(sales)))
```

EXAMPLE 3: EMPLOYEE SALARY REPORT

```
employees = {
    "Rahul": 50000,
    "Amit": 60000
}

with open("salary_report.txt", "w") as file:

    for name, salary in employees.items():
        file.write(f"{name} : {salary}\n")
```

EXAMPLE 4: READ CUSTOMER LIST

```
with open("customers.txt", "r") as file:

    customers = file.readlines()

print(customers)
```

Mini Project 1: Student File Creator

```
name = input("Enter Student Name: ")

with open("students.txt", "a") as file:

    file.write(name + "\n")

print("Student Saved")
```

Mini Project 2: Sales Report Generator

```
sales = [12000, 25000, 18000]

with open("sales_report.txt", "w") as file:
```

```
    file.write("Total Sales = " + str(sum(sales)))
```

Mini Project 3: Employee Database

```
employees = [
    "Rahul",
    "Amit",
    "Priya"
]

with open("employees.txt", "w") as file:

    for emp in employees:
        file.write(emp + "\n")
```

Mini Project 4: Read CSV Employee Data

```
import csv

with open("employees.csv", "r") as file:

    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Practice Questions

BASIC

1. Create a text file.
2. Read a text file.
3. Read one line.
4. Read all lines.
5. Close a file.

WRITING FILES

6. Write a student name.

7. Write employee details.
8. Write sales report.
9. Append customer names.
10. Save monthly sales.

CSV

11. Read CSV.
12. Write CSV.
13. Print CSV rows.
14. Count rows.
15. Display employee salaries.

EXCEPTION HANDLING

16. Handle missing file.
17. Check if file exists.
18. Delete a file.
19. Create a new file.
20. Append data safely.

BUSINESS PROBLEMS

21. Employee report.
22. Customer database.
23. Sales report.
24. Attendance report.
25. Product inventory.
26. Invoice report.
27. Profit report.
28. Expense report.
29. Log file reader.
30. Store KPI results.

Assignment (Data Analyst Level)

Create an Employee Report Generator.

Employee data:

```
employees = [
    ("Rahul", 50000),
    ("Amit", 60000),
    ("Priya", 55000)
]
```

Tasks:

1. Save all employee details to **employee_report.txt**.
2. Calculate:
    - Total Salary
    - Average Salary
    - Highest Salary
    - Lowest Salary
3. Save these results at the end of the report.

Expected file output:

```
Employee Report

Rahul : 50000
Amit : 60000
Priya : 55000

----------------------

Total Salary : 165000
Average Salary : 55000
Highest Salary : 60000
Lowest Salary : 50000
```

Interview Questions

1. What is file handling?
2. Why is file handling important?
3. What are different file modes?
4. Difference between `r`, `w`, and `a`?
5. What does `read()` do?
6. Difference between `read()`, `readline()`, and `readlines()`?
7. Why should you use the `with` statement?

8. How do you check if a file exists?
9. How do you handle `FileNotFoundError`?
10. What is a CSV file?
11. How do you read a CSV file in Python?
12. Why is CSV widely used in Data Analysis?
13. What is the difference between text files and CSV files?
14. How do you write data into a CSV file?
15. Give a real-world example of file handling in a Data Analyst role.

Learning Outcome

After Day 9, you should be able to:

✅ Read and write text files

✅ Append data to existing files

✅ Use the `with` statement safely

✅ Handle file-related exceptions

✅ Read and write CSV files using the `csv` module

✅ Generate reports and automate file-based tasks used in Data Analysis